

SPARK — DAY 18 IMPLEMENTATION REPORT

APPLICATION PROJECT FOUNDATION & VIDEO UPLOAD ARCHITECTURE

Project: SPARK Badminton Shot Recognition	Date: 18 September 2026
Repository: spark-badminton-shot-recognition	Phase: Application Development Foundation
Team: Sirish Chandra, Priyanshu, Ashwidha, Thakur Swetan Singh, Kaustub	Status: Completed & Verified

1. Executive Summary & Scope Boundary

Day 18 marks the official transition from core ML research to full-stack application development. The objective was to create a clean, modern, production-grade application project structure for SPARK, establishing the frontend foundation, the backend API foundation, and the end-to-end video ingestion and preview workflow without touching or altering existing research checkpoints or datasets.

Key Principle: Strictly zero cleanup and zero modification of existing research assets. The existing research folders (including EXP24 multimodal transformer-BiLSTM checkpoints, ShuttleSet annotations, and ResNet spatial feature caches in D:\PS_DATA) remain completely untouched as reference assets.

2. Application Directory Layout

The application is housed in a clean, self-contained project root (D:\PS_DATA\SPARK) connected directly to the GitHub remote repository:

```
SPARK/
■■■■ frontend/ # Next.js 16 + React 19 + TypeScript + Tailwind CSS
■ ■■■■ app/ # App Router (page.tsx, layout.tsx, globals.css)
■ ■■■■ components/ # UI Components (Header, Uploader, Preview, Results)
■ ■■■■ lib/ # API Client (health check, multipart upload)
■ ■■■■ types/ # TypeScript interfaces & classification definitions
■ ■■■■ .env.example # Configurable backend URL template
■■■■ backend/ # FastAPI + Uvicorn + Pydantic
■ ■■■■ app/
■ ■ ■■■■ main.py # FastAPI application, CORS, root & health routes
■ ■ ■■■■ api/video.py # Secure multipart video upload router
■ ■ ■■■■ core/config.py# Dynamic configuration (MAX_UPLOAD_SIZE_MB)
■ ■ ■■■■ schemas/ # Pydantic response models
■ ■ ■■■■ services/ # ML inference boundary (Day 19 placeholder)
■ ■ ■■■■ utils/ # Security, sanitization & traversal defenses
■ ■■■■ storage/uploads/ # Secure local uploaded video repository
■■■■ model/ # Architecture boundary documentation & checkpoints
■■■■ data/ # Small runtime fixtures (large raw videos gitignored)
■■■■ docs/ # Implementation reports & technical specifications
■■■■ results/ # Output analytics & evaluation metrics
■■■■ tests/ # Automated Pytest suite (8/8 passing)
■■■■ .gitignore # Comprehensive rules (ignoring media, weights, venv)
■■■■ README.md # Complete scientific & deployment documentation
```

3. Backend Implementation & API Specifications

The backend is developed using **FastAPI**, **Uvicorn**, and **Pydantic**. It exposes modular, production-ready endpoints with automatic OpenAPI documentation at /docs.

Method	Endpoint	Status	Description & Contract
GET	/	200 OK	Returns SPARK system identification, status, and interactive docs URL.
GET	/health	200 OK	Returns {"status": "ok", "project": "SPARK", "service": "badminton-shot-recognition-api"}.
POST	/api/video/upload	201 Created	Accepts multipart video (.mp4, .mov, .avi, .mkv, .webm), validates size against configurable MAX_UPLOAD_SIZE_MB, generates UUID, saves safely, returns video_id.

4. Upload Security Protocols & Hardening

Because video uploads can pose security and resource exhaustion risks, the SPARK backend incorporates strict defenses:

- **Filename Sanitization:** Discards directory paths, null bytes, and shell metacharacters, eliminating path traversal risks (e.g. ../etc/passwd).

- **UUID Server-Side Storage:** Uploaded files are persisted as {video_id}_{sanitized_name}, preventing file overwriting and name collisions.
- **Format Whitelisting:** Strictly allows .mp4, .mov, .avi, .mkv, .webm. Executable and script extensions are rejected immediately with HTTP 400.
- **Configurable File Size Limit:** Configured via MAX_UPLOAD_SIZE_MB (default: 200MB, easily increased for full match files via environment variable). Streaming chunked disk writes abort and delete partial files if size exceeds the threshold, returning HTTP 413.
- **Internal Path Obfuscation:** Raw disk storage paths are never leaked in client responses; clients only receive the sanitized filename and generated video_id.

5. Frontend Architecture & User Interface

The frontend is implemented with **Next.js** (App Router), **React 19**, and **TypeScript**, adhering to a dark premium sports-technology aesthetic:

- **Brand Identity:** High-contrast obsidian slate background (#080C14) paired with emerald badminton court accents and subtle court grid lines.
- **Real-Time Service Health:** Header incorporates an asynchronous heartbeat pill dynamically displaying whether the FastAPI backend is Online or Offline.
- **Badminton Taxonomy (5 Classes):** Interactive reference cards for **SMASH**, **CLEAR**, **DROP**, **DRIVE**, and **NET SHOT** based on the verified EXP24 taxonomy.
- **Drag-and-Drop Video Ingestion:** Supports both native file browser selection and drag-and-drop with real-time format validation.
- **Interactive HTML5 Video Preview:** Instant player preview with metadata inspection (file name, formatted byte size, MIME type) before initiating processing.
- **Authentic Prediction Placeholders:** In strict compliance with research ethics, prediction cards display genuine placeholders (—) without fabricating confidence or class labels.

6. Automated Testing & Verification

A comprehensive Pytest automated test suite (tests/test_backend.py) validates all backend endpoints and edge cases:

Test Case	Target Functionality	Result
test_root_endpoint	Validates service branding, title, and OpenAPI docs link	PASSED
test_health_endpoint	Validates exact JSON contract: status, project, service	PASSED
test_video_upload_valid_mp4	Validates multipart stream, 201 Created, UUID, and disk persistence	PASSED
test_video_upload_other_supported_extensions	Validates .mov, .avi, .mkv, and .webm handling	PASSED
test_video_upload_invalid_extension_rejected	Ensures .exe, .txt, .pdf are rejected with HTTP 400	PASSED
test_video_upload_missing_file	Validates empty payload handling with HTTP 422	PASSED

test_filename_sanitization_and_traversal_defense	Neutralizes directory traversal (../..) and special characters	PASSED
test_unique_video_id_generation	Confirms uniqueness across concurrent video uploads	PASSED

7. Next Milestone: Day 19 ML Model Integration

With the application and video ingestion foundation established and verified, Day 19 will connect the verified EXP24 Multimodal Transformer-BiLSTM checkpoint to the application layer:

- 1. **Inference Boundary Implementation:** Flesh out backend/app/services/inference_service.py to execute OpenCV video frame extraction and ResNet-18 spatial feature generation.
- 2. **Checkpoint Loading:** Safely reference the existing EXP24 model checkpoint without modifying the original training artifacts.
- 3. **Results Pipeline:** Wire the frontend results component to receive real softmax probabilities, class predictions, and latency metrics.

Prepared by: SPARK Development Team

Approved for: Day 18 Application Milestone